

async / await ile beklerken kilitlenme

Uzun süren işlemler devam ederken uygulamanın başka işlere cevap verebilmesini öğren. Önce çalışan örnekler, sonra senin tamamlayacağın iskeletler.

Yaklaşık	Odak	Çıktı
120–180 dk	Task ve await	Asenkron görev simülasyonu

Bugünün çalışma kuralı

Her örnekte ekrana yazılan satırların sırasını çalıştırmadan önce tahmin et. Sonra programı çalıştır ve tahmininle karşılaştır.

Bitiş ölçütü

- Senkron ve asenkron çalışma farkını günlük dille açıklayabilmek
- Task, async ve await rollerini ayırt edebilmek
- Task ve Task<T> döndüren metot yazabilmek
- İki bağımsız işi Task.WhenAll ile birlikte bekleyebilmek

1. Neden asenkron programlama?

Dosya okuma, HTTP isteği veya veritabanı sorgusu gibi işlemler zaman alabilir. Program bu sırada işlemciyi boşuna meşgul etmek yerine beklemeyi yönetebilir.

Günlük örnek

Kahve makinesi çalışırken başında beklemek yerine tost hazırlarsın. Kahve hazır olduğunda kaldığın yerden devam edersin. Asenkronluk aynı anda daha hızlı işlemci kullanmak değil, bekleme süresini verimli yönetmektir.

Senkron bekleme

```
Console.WriteLine("Başladı");  
Thread.Sleep(2000);  
Console.WriteLine("Bitti");
```

Bu iki saniye boyunca çalışan thread bloke olur.

Asenkron bekleme

```
Console.WriteLine("Başladı");  
await Task.Delay(2000);  
Console.WriteLine("Bitti");
```

Task.Delay gerçek bir işi taklit etmek için kullanılır. await, görev tamamlanınca devam edilmesini söyler.

Önemli

Task.Delay eğitimde beklemeyi simüle eder. Gerçek uygulamada beklenen işlem HTTP, dosya veya veritabanı çağırısı olabilir.

2. Task, async ve await

Kavram	Rolü
Task	Devam eden veya ileride tamamlanacak işi temsil eder.
async	Metot içinde await kullanılabileceğini belirtir.
await	Task tamamlanınca sonraki satırdan devam eder.

Küçük çalışan metot

```
static async Task MesajYazAsync()
{
    Console.WriteLine("İşlem başladı");
    await Task.Delay(1000);
    Console.WriteLine("İşlem tamamlandı");
}

await MesajYazAsync();
```

İsimlendirme

Asenkron metot adlarının sonuna Async eklemek yaygın bir C# geleneğidir; metodun beklenebilir olduğunu anlatır.

Görev 1 — Bekleyen sayaç

```
static async Task BekleAsync()
{
    Console.WriteLine("Bekleme başladı");

    // 2 saniyelik asenkron bekleme ekle.
    await /* ... */;

    Console.WriteLine("Bekleme tamamlandı");
}

// Metodu await ederek çağır.
/* ... */;
```

3. Task<T>: Sonuç döndüren görev

Normal bir metod int döndürüyorsa asenkron karşılığı çoğunlukla Task<int> döndürür.

Küçük çalışan örnek

```
static async Task<int> SayiGetirAsync()
{
    await Task.Delay(1000);
    return 42;
}

int sonuc = await SayiGetirAsync();
Console.WriteLine(sonuc);
```

Senkron	Asenkron
void	Task
int	Task<int>
string	Task<string>

Görev 2 — Kullanıcı adı getir

```
static async Task<string> KullaniciGetirAsync()
{
    // 1 saniye bekle.
    await /* ... */;

    // "Kaan" değerini döndür.
    return /* ... */;
}

string kullanıcı = await /* metodu çağır */;
Console.WriteLine($"Hoş geldin, {kullanıcı}");
```

4. await akışı nasıl etkiler?

await programın tamamını kapatmaz. İçinde bulunduğu asenkron metodun devamını görev tamamlanana kadar erteler.

Çıktı sırasını tahmin et

```
Console.WriteLine("A");  
Task bekleme = Task.Delay(1500);  
Console.WriteLine("B");  
await bekleme;  
Console.WriteLine("C");
```

Çıktı

A B yaklaşık 1,5 saniye bekleme C. Task oluşturulduğunda başlar; await satırında tamamlanması beklenir.

await unutulursa

```
static async Task IslemAsync()  
{  
    await Task.Delay(1000);  
    Console.WriteLine("İşlem bitti");  
}  
  
IslemAsync(); // Uyarı: görev beklenmiyor  
Console.WriteLine("Program devam ediyor");
```

Görev beklenmezse çağıran kod devam eder; uygulama görev tamamlanmadan kapanabilir ve hatalar gözden kaçabilir.

Kural

Bir Task sonucunu bilerek arka planda bırakmıyorsan çoğunlukla await et.

Görev 3

Yukarıdaki örneği await kullanarak düzelt ve çıktı sırasını çalıştırmadan önce tahmin et.

5. Asenkron hataları yakalamak

await edilen Task hata verirse exception, await satırında ortaya çıkar ve normal try/catch ile yönetilebilir.

Küçük çalışan örnek

```
static async Task DosyaTaklidiAsync()
{
    await Task.Delay(500);
    throw new InvalidOperationException("Dosya okunamadı.");
}

try
{
    await DosyaTaklidiAsync();
}
catch (InvalidOperationException ex)
{
    Console.WriteLine($"Hata: {ex.Message}");
}
```

Bağlantı

Gün 4'te öğrendiğin try/catch yapısı değişmedi. Yeni olan, hatanın Task tamamlanırken ortaya çıkmasıdır.

Görev 4 — Güvenli çağrı

```
static async Task VeriGetirAsync()
{
    await Task.Delay(500);
    throw new Exception("Sunucuya ulaşılamadı.");
}

// Metodu try içinde await et.
// Exception'ı catch içinde yakala.
// ex.Message değerini ekrana yazdır.
try
{
    /* ... */
}
catch (Exception ex)
{
    /* ... */
}
```

6. Bağımsız işleri birlikte beklemek

Birbirine bağlı olmayan görevleri önce başlatıp sonra Task.WhenAll ile birlikte bekleyebiliriz.

Küçük çalışan örnek

```
static async Task IslemAsync(string ad, int sure)
{
    Console.WriteLine($"{ad} başladı");
    await Task.Delay(sure);
    Console.WriteLine($"{ad} bitti");
}

Task birinci = IslemAsync("Birinci", 1500);
Task ikinci = IslemAsync("İkinci", 1000);

await Task.WhenAll(birinci, ikinci);
Console.WriteLine("Tüm işlemler tamamlandı");
```

Neden yaklaşık 1,5 saniye?

İki bağımsız bekleme birlikte ilerler. Toplam süre yaklaşık 2,5 saniye değil, en uzun görev kadar olur.

Görev 5 — İki indirme taklidi

```
// IndirAsync adında Task döndüren bir metod yaz.
// Parametreler: dosyaAdı ve süre.
// Başlangıç mesajı yaz, süre kadar bekle, bitiş mesajı yaz.

Task dosya1 = /* 1200 ms süren çağrı */;
Task dosya2 = /* 800 ms süren çağrı */;

// İkisini birlikte bekle.
await /* ... */;

Console.WriteLine("İndirmeler tamamlandı");
```

7. Final uygulama ve kapanış

Sipariş hazırlama sürecini asenkron olarak taklit et. Tam çözüm öğrenci kodu görülmeden verilmez.

Görev adımları

- SiparişHazırlaAsync metodu Task<string> döndürsün.
- Metot sipariş numarası ve bekleme süresi alsın.
- Başlangıç mesajından sonra Task.Delay ile beklesin.
- Tamamlanınca sipariş numarasını içeren bir string döndürsün.
- İki siparişi başlat ve Task.WhenAll ile birlikte bekle.
- Sonuçları foreach ile yazdır; çağrıyı try/catch içine al.

```
static async Task<string> SiparisHazirlaAsync(
    int siparisNo,
    int sure)
{
    Console.WriteLine($"Sipariş {siparisNo} hazırlanıyor");

    await /* bekleme */;

    return /* tamamlanma mesajı */;
}

try
{
    Task<string> siparis1 = /* çağrı */;
    Task<string> siparis2 = /* çağrı */;

    string[] sonuclar = await /* birlikte bekle */;

    foreach (string sonuc in sonuclar)
    {
        /* yazdır */
    }
}
catch (Exception ex)
{
    /* hata mesajı */
}
```

Mülakat için kısa notlar

- async tek başına yeni thread oluşturmaz.
- await, Task tamamlanınca metodun devam etmesini sağlar.
- Task değer döndürmeyen; Task<T> değer döndüren asenkron işi temsil eder.
- Bağımsız işler Task.WhenAll ile birlikte beklenebilir.

Proje ve Git

```
cd "C:\Users\Administrator\.NetEgitim\DotNetIlkHafta"
mkdir Gun6-AsyncAwait
cd Gun6-AsyncAwait
dotnet new console
code .

cd "C:\Users\Administrator\.NetEgitim"
git add .
git commit -m "Gün 6 async await alıştırmaları tamamlandı"
git push
```

Net kapanış

Final uygulama üç farklı süreyle çalışıyorsa ve çıktı sırasını açıklayabiliyorsan Gün 6 tamamdır.